



Universidade do Minho

UNIVERSIDADE DO MINHO

LICENCIATURA EM CIÊNCIAS DA COMPUTAÇÃO

Trabalho Prático 2

Processamento de Linguagens e Compiladores

Grupo 8

Cláudia Rego Faria
(A105531)

Loren Narrane Leandro Oliveira da Silva
(A102514)

Patrícia Daniela Fernandes Bastos
(A102502)

12 de janeiro de 2025

Conteúdo

1	Introdução	3
2	Enunciado	4
3	Concepção da Solução	5
3.1	Sintaxe da Linguagem PLC	5
3.1.1	Declaração e atribuição de variáveis	5
3.1.2	Operadores de comparação	5
3.1.3	Operações numéricas	6
3.1.4	Operadores lógicos	6
3.1.5	Instruções de seleção	6
3.1.6	Ciclo while	6
3.1.7	Definição e invocação de subprogramas	6
3.1.8	Input/Output	7
3.2	Símbolos	7
3.3	Desenho da GIC	8
4	Exemplos de funcionamento	10
4.0.1	Exemplo 1	10
4.0.2	Exemplo 2	11
4.0.3	Exemplo 3	11
4.0.4	Exemplo 4	12
4.0.5	Exemplo 5	14
4.0.6	Exemplo 6	14
4.0.7	Exemplo 7	16
5	Conclusão	17

Capítulo 1

Introdução

Este trabalho foi proposto no âmbito da disciplina de Processamento de Linguagens e Compiladores e consiste no desenvolvimento de uma linguagem de programação imperativa que visa a aplicação dos conhecimentos adquiridos na disciplina. A linguagem é acompanhada de um compilador que transforma programas desta linguagem em código Assembly da Máquina Virtual EWVM estudada nas aulas.

O seguinte relatório contém a explicação da gramática e da sintaxe e programas para teste da linguagem e da sua conversão para Assembly.

Devido ao amor partilhado pelo nosso grupo pela gastronomia portuguesa, a linguagem foi batizada de Rissol. O Rissol é simples, eficaz e delicioso. Esperamos que o leitor goste tanto do Rissol como o nosso grupo gostou de o desenvolver.

Capítulo 2

Enunciado

Pretende-se que comece por definir uma linguagem de programação imperativa simples, a seu gosto. Apenas deve ter em consideração que essa linguagem terá de permitir:

- *declarar* variáveis atômicas do tipo *inteiro*, com os quais se podem realizar as habituais operações aritméticas, relacionais e lógicas;
- *efetuar* instruções algorítmicas básicas como a *atribuição do valor de expressões numéricas a variáveis*;
- *ler* do *standard input* e *escrever* no *standard output*;
- *efetuar* instruções de *seleção* para controlo do fluxo de execução;
- *efetuar* instruções de repetição (*cíclicas*) para controlo do fluxo de execução, permitindo o seu aninhamento.
Note que deve implementar pelo menos o ciclo **while-do**, **repeat-until** ou **for-do**.

Adicionalmente deve ainda suportar, à sua escolha, uma das duas funcionalidades seguintes:

- *declarar e manusear* variáveis estruturadas do tipo *array* (*a 1 ou 2 dimensões*) de *inteiros*, em relação aos quais é apenas permitida a operação de indexação (índice inteiro);
- *definir e invocar subprogramas* sem parâmetros mas que possam retornar um resultado do tipo inteiro.

Como é da praxe neste tipo de linguagens, as variáveis deverão ser declaradas no início do programa e não pode haver re-declarações, nem utilizações sem declaração prévia. Se nada for explicitado, o valor da variável após a declaração é 0 (zero). O compilador deve gerar **pseudo-código**, Assembly da Máquina Virtual VM.

Capítulo 3

Concepção da Solução

Para criar a linguagem, começámos por escrever frases de exemplo que fossem reconhecidas por ela. Depois, escrevemos a gramática, formalizando assim a linguagem. Por fim, implementámos, em Python, o analisador léxico e o parser. As regras de tradução para Assembly encontram-se no parser.

Acordámos que a linguagem incluiria a definição e implementação de sub-programas, sem parâmetros mas que retornam um inteiro, bem como a estrutura de controlo while-do.

3.1 Sintaxe da Linguagem PLC

3.1.1 Declaração e atribuição de variáveis

```
1
2     int x
3     int y
4     int z
5
6     x = 1
7     y = 2
8     z = 3
9
```

3.1.2 Operadores de comparação

```
1
2     x > y
3     x < y
4     x >= y
5     x <= y
6     x != y
7     x == y
8
```

3.1.3 Operações numéricas

```
1
2     x + y
3     x - y
4     x / y
5     x * y
6     x % y
7
```

3.1.4 Operadores lógicos

```
1
2     x and y
3     x or y
4     ! x
5
```

3.1.5 Instruções de seleção

```
1
2     if CONDICAO:
3         ...
4     else:
5         ...
6     endelse
7     endif
8
9     if CONDICAO:
10        ...
11    endif
12
```

3.1.6 Ciclo while

```
1
2     while CONDICAO:
3         ...
4     endwhile
5
```

3.1.7 Definição e invocação de subprogramas

```
1
2     define funcao():
3         ...
4         return VAR
5     enddefine
```

```

6
7     funcao()
8
9     x = funcao()
10    y = funcao() + x
11

```

3.1.8 Input/Output

```

1
2     print("Escreve o valor de x")
3     x = input
4     print(x)
5

```

3.2 Símbolos

Os símbolos da linguagem são os seguintes:

```

'AND',
'OR',
'LPAREN',
'RPAREN',
'DOISPT',
'DIV',
'MOD',
'IGUAL',
'MAIS',
'MENOS',
'VEZES',
'IF',
'ENDIF',
'ELSE',
'ENDELSE',
'MAIOR',
'MENOR',
'DEFINE',
'ENDDEFINE',
'WHILE',
'ENDWHILE',
'RETURN',
'INT',
'NAO',
'INPUT',
'PRINT',

```

```
'STRING',  
'NUM',  
'PALAVRA'
```

3.3 Desenho da GIC

A nossa linguagem é gerada pela seguinte grámatica independente de contexto:

```
Programa : Declaracoes Funcoes  
  
Declaracoes : Declaracao  
              | Declaracoes Declaracao  
  
Funcoes : Funcao  
          | Funcoes Funcao  
  
Funcao : Selecao  
        | Ciclo  
        | Subprograma  
        | Atribuicao  
        | Print  
        | ChamadaFuncao  
  
Declaracao : INT Var  
  
Atribuicao : Var IGUAL Expressao  
           | Var IGUAL INPUT  
  
Expressao : Operacao  
           | Num  
           | Var  
           | ChamadaFuncao  
  
Operacao : Expressao Operador Expressao  
  
Operador : VEZES  
          | MAIS  
          | MENOS  
          | DIV  
          | MOD
```



```

Comparador : MAIOR
            | MENOR
            | MAIOR IGUAL
            | MENOR IGUAL
            | NAO IGUAL
            | IGUAL IGUAL

Selecao : IF Condicao DOISPT Corpo ELSE DOISPT Corpo ENDELSE ENDIF
        | IF Condicao DOISPT Corpo ENDIF

Ciclo : WHILE Condicao DOISPT Corpo ENDWHILE

Condicao : LPAREN Expressao Comparador Expressao RPAREN
        | LPAREN Condicao AND Condicao RPAREN
        | LPAREN Condicao OR Condicao RPAREN
        | LPAREN NAO Condicao RPAREN
        | LPAREN Condicao RPAREN

Subprograma : DEFINE F DOISPT Corpo RETURN Expressao ENDDEFINE

Corpo : C Corpo
      | C

C : Selecao
  | Atribuicao
  | Ciclo
  | ChamadaFuncao
  | Print

F : PALAVRA LPAREN RPAREN

ChamadaFuncao : F

VAR : PALAVRA

Print : PRINT LPAREN STRING RPAREN
      | PRINT LPAREN VAR RPAREN

```

Capítulo 4

Exemplos de funcionamento

4.0.1 Exemplo 1

```
1
2      int x
3      int y
4
5      x = 10
6      y = 20
7
8      define fun():
9          x = x - 1
10         return x
11     enddefine
12
13     if (x < y):
14         fun()
15     endif
16
```

Código Assembly gerado:

```
1      PUSHI 0
2      PUSHI 0
3      START
4      PUSHI 10
5      STOREG 0
6      PUSHI 20
7      STOREG 1
8      PUSHG 0
9      PUSHG 1
10     INF
11     JZ label0
12     PUSHA fun
13     CALL
14     label0: NOP
15     STOP
16
```

```

17     fun:
18     PUSHG 0
19     PUSHI 1
20     SUB
21     STOREG 0
22     PUSHG 0
23     RETURN
24

```

4.0.2 Exemplo 2

```

1
2     int x
3     int y
4
5     x = 10
6     y = 20
7
8     while (x < y):
9         x = x + 1
10    endwhile
11

```

Código Assembly gerado:

```

1
2     PUSHI 0
3     PUSHI 0
4     START
5     PUSHI 10
6     STOREG 0
7     PUSHI 20
8     STOREG 1
9     label0c: NOP
10    PUSHG 0
11    PUSHG 1
12    INF
13    JZ label0f
14    PUSHG 0
15    PUSHI 1
16    ADD
17    STOREG 0
18    JUMP label0c
19    label0f: NOP
20    STOP
21

```

4.0.3 Exemplo 3

```

1
2  int x
3
4  define paridade():
5      if (x % 2 == 0):
6          print("par")
7      else:
8          print("impar")
9      endelse
10     endif
11     return x
12 enddefine
13
14 x = input
15 paridade()
16

```

Código Assembly gerado:

```

1
2  PUSHI 0
3  START
4  READ
5  ATOI
6  STOREG 0
7  PUSHA paridade
8  CALL
9  STOP
10 paridade:
11  PUSHG 0
12  PUSHI 2
13  MOD
14  PUSHI 0
15  EQUAL
16  JZ label0
17  PUSHS "par"
18  WRITES
19  WRITELN
20  JUMP label0f
21  label0: NOP
22  PUSHS "impar"
23  WRITES
24  WRITELN
25  label0f: NOP
26  PUSHG 0
27  RETURN
28

```

4.0.4 Exemplo 4

```

1
2  int x

```

```

3  int y
4  int z
5
6  print("Escreve o valor de x")
7  x = input
8  print("Escreve o valor de y")
9  y = input
10 if (x == y):
11     z = x + y
12 else:
13     z = y
14 endelse
15 endif
16 print(z)
17

```

Código Assembly gerado:

```

1
2  PUSHI 0
3  PUSHI 0
4  PUSHI 0
5  START
6  PUSHS "Escreve o valor de x"
7  WRITES
8  WRITELN
9  READ
10 ATOI
11 STOREG 0
12 PUSHS "Escreve o valor de y"
13 WRITES
14 WRITELN
15 READ
16 ATOI
17 STOREG 1
18 PUSHG 0
19 PUSHG 1
20 EQUAL
21 JZ label0
22 PUSHG 0
23 PUSHG 1
24 ADD
25 STOREG 2
26 JUMP label0f
27 label0: NOP
28 PUSHG 1
29 STOREG 2
30 label0f: NOP
31 PUSHG 2
32 WRITEI
33 WRITELN
34 STOP
35

```

4.0.5 Exemplo 5

```
1
2  int idade
3
4  print("Qual a sua idade?")
5  idade = input
6  if (idade >= 18):
7      print("Pode tirar a carta!")
8  else:
9      print("Espere mais um pouco.")
10 endelse
11 endif
12
```

Código Assembly gerado:

```
1
2  PUSHI 0
3  START
4  PUSHS "Qual a sua idade?"
5  WRITES
6  WRITELN
7  READ
8  ATOI
9  STOREG 0
10 PUSHG 0
11 PUSHI 18
12 SUPEQ
13 JZ label0
14 PUSHS "Pode tirar a carta!"
15 WRITES
16 WRITELN
17 JUMP label0f
18 label0: NOP
19 PUSHS "Espere mais um pouco."
20 WRITES
21 WRITELN
22 label0f: NOP
23 STOP
24
```

4.0.6 Exemplo 6

```
1
2  int a
3  int b
4  int temp
5  int result
6
7  define mdc():
8      a = input
```

```

9      b = input
10
11     while (b != 0) :
12         temp = b
13         b = a % b
14         a = temp
15     endwhile
16
17     return a
18 enddefine
19
20 result = mdc()
21 print("MDC: ")
22 print(result)
23

```

Código Assembly gerado:

```

1
2      PUSHI 0
3      PUSHI 0
4      PUSHI 0
5      PUSHI 0
6      START
7      PUSHA mdc
8      CALL
9      STOREG 3
10     PUSHS "MDC: "
11     WRITES
12     WRITELN
13     PUSHG 3
14     WRITEI
15     WRITELN
16     STOP
17     mdc:
18     READ
19     ATOI
20     STOREG 0
21     READ
22     ATOI
23     STOREG 1
24     label0c: NOP
25     PUSHG 1
26     PUSHI 0
27     EQUAL
28     NOT
29     JZ label0f
30     PUSHG 1
31     STOREG 2
32     PUSHG 0
33     PUSHG 1
34     MOD
35     STOREG 1

```

```

36     PUSHG 2
37     STOREG 0
38     JUMP label0c
39     label0f: NOP
40     PUSHG 0
41     RETURN
42

```

4.0.7 Exemplo 7

```

1
2     int x
3     int y
4
5     y = x + 1
6
7     print(y)
8

```

Código Assembly gerado:

```

1
2     PUSHI 0
3     PUSHI 0
4     START
5     PUSHG 0
6     PUSHI 1
7     ADD
8     STOREG 1
9     PUSHG 1
10    WRITEI
11    WRITELN
12    STOP
13

```


Capítulo 5

Conclusão

Este projeto permitiu-nos aplicar de forma prática os conhecimentos adquiridos nas aulas de Processamento de Linguagens e Compiladores. O desenvolvimento de um compilador que gera pseudo-código Assembly para uma máquina virtual proporcionou-nos a oportunidade de relembrar o funcionamento das linguagens de máquina, especialmente na implementação de jumps e operações na stack.

Ao longo do trabalho, aprofundamos o nosso entendimento sobre os módulos Lex e Yacc, enfrentando desafios ao traduzir uma linguagem de alto nível para uma de baixo nível.

Cumprimos todos os objetivos propostos para o trabalho prático e, como implementação adicional, optamos pela definição e invocação de subprogramas. Consideramos que o resultado final reflete um bom domínio das ferramentas apresentadas e dos conceitos ensinados, servindo como uma base sólida para desafios futuros.